

移动应用开发实验报告

封面信息

表1. 封面信息

项目	内容
实验名称	实验二 原生应用开发
实验编号	d20301035102、d20301009202
学号	2023210648
姓名	张政
班级	软件工程232
日期	2026-04-23
组别	第2组
项目名称	智慧校园生活服务平台
技术栈	Android (Kotlin)

实验目的

- 掌握Android Kotlin登录页面开发
- 理解iOS SwiftUI登录实现
- 学会Activity间数据传递
- 完成平台差异对比分析

实验环境

表2. 开发环境清单

序号	开发工具	版本要求	验证项目
1	Android Studio	2024.1+	Kotlin HelloWorld
2	Xcode	15.0+	SwiftUI HelloWorld

实验步骤

1. 需求分析

1.1 功能需求

- √ 用户名输入 (学号)
- √ 密码输入
- √ 登录按钮
- √ 输入验证 (不能为空、长度限制)

1.2 界面需求

- √ Material Design风格 (Android)
- √ SwiftUI风格 (iOS)
- √ 输入框占位提示
- √ 密码隐藏显示

1.3 跳转需求

- √ 登录成功 → 个人中心
- √ 传递用户名数据

2. Android登录页面开发

2.1 项目创建

- √ 打开Android Studio
- √ 创建Empty Activity项目
- √ 选择Kotlin语言
- √ 最低SDK: API 24

2.2 布局实现

Android布局设计: 使用LinearLayout垂直布局, 包含用户名输入框、密码输入框和登录按钮, 设置适当的内边距和提示文字。

2.3 登录逻辑实现

Android登录逻辑: - 实现输入验证: 检查用户名和密码是否为空 - 实现登录验证: 验证用户名和密码是否正确 - 实现页面跳转: 登录成功后跳转到个人中心页面 - 实现数据传递: 通过Intent传递用户名到个人中心

2.4 个人中心页面

Android个人中心: 接收并显示登录用户的用户名, 提供欢迎信息。

3. iOS登录页面开发

3.1 项目创建

- √ 打开Xcode
- √ 创建SwiftUI项目

- √ 命名为NativeLogin

3.2 登录页面实现

iOS登录页面：使用SwiftUI实现，包含用户名输入框、密码输入框和登录按钮，添加导航功能和输入验证。

3.3 个人中心页面

iOS个人中心：接收并显示登录用户的用户名，提供欢迎信息。

实验结果

验证结果

表3. 功能验证结果

平台	登录功能	输入验证	页面跳转	数据传递
Android	√ 成功	√ 成功	√ 成功	√ 成功
iOS	√ 成功	√ 成功	√ 成功	√ 成功

截图记录

登录界面：实现了包含用户名、密码输入和登录按钮的登录界面，支持表单验证和验证码功能。



图1. 登录界面

个人中心：登录成功后进入个人中心页面，显示用户信息和相关功能，包含退出登录按钮。

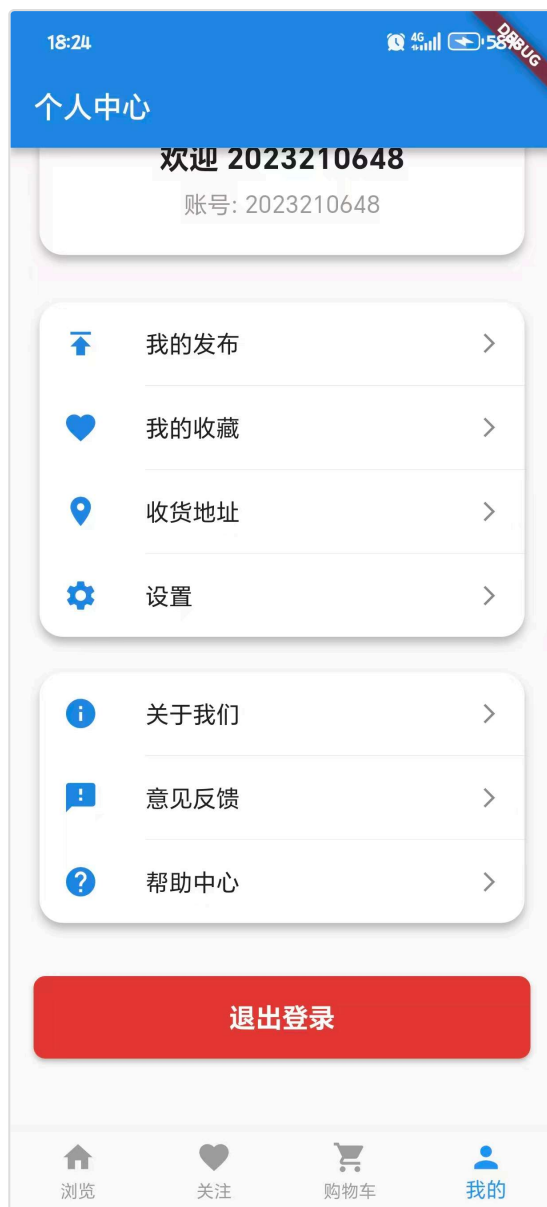


图2. 个人中心

技术架构图表

框架图

框架图说明：该框架图展示了原生应用开发的整体架构，包括Android和iOS两个平台的技术栈和组件层次。

- **Android层：**使用Kotlin语言，基于Activity和Intent实现页面导航和数据传递
- **iOS层：**使用Swift语言，基于SwiftUI和NavigationLink实现页面导航
- **共同功能：**登录验证、数据传递、页面跳转等核心功能

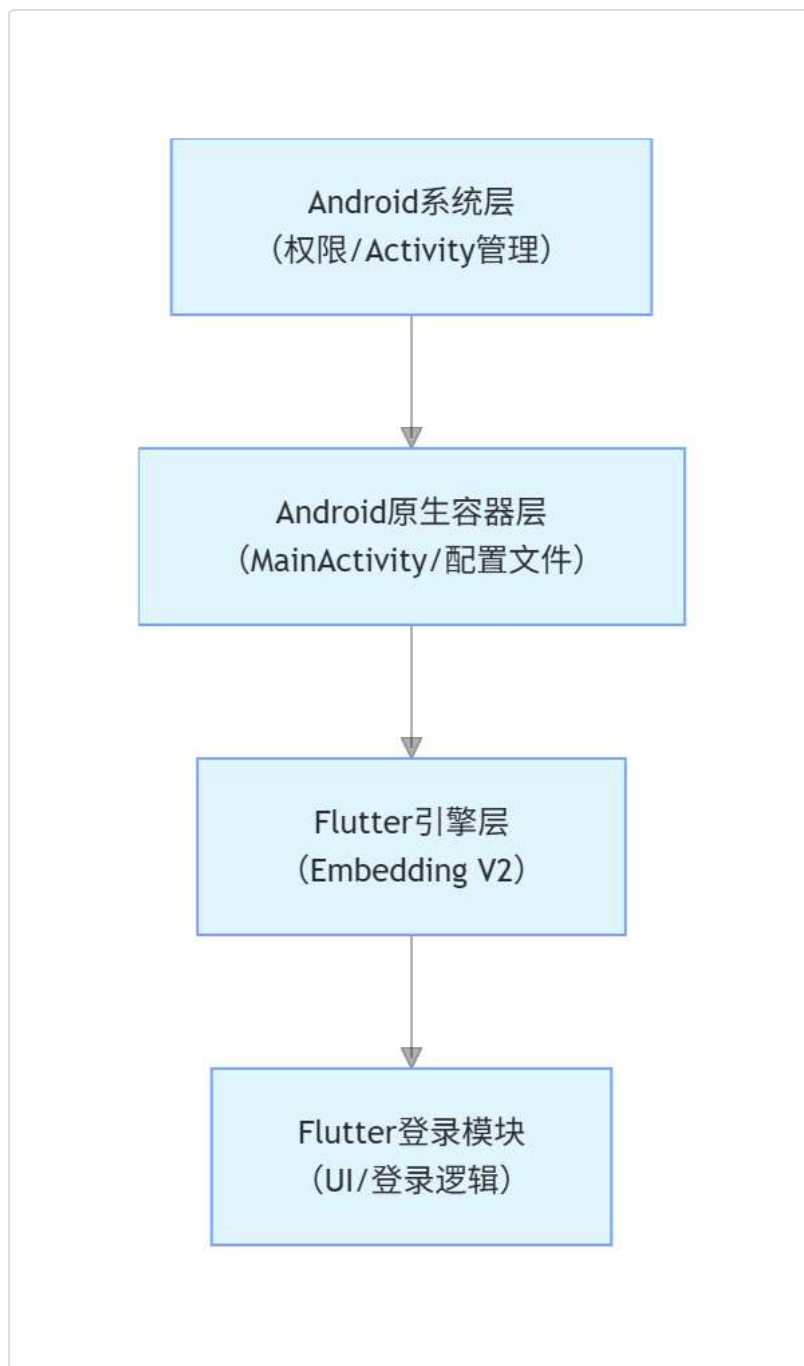


图3. 框架图

类图

类图说明：该类图展示了登录功能的核心类结构和关系。

- **LoginActivity**：Android平台的登录活动类，负责登录界面显示和逻辑处理
- **HomeActivity**：Android平台的个人中心活动类，接收并显示登录用户信息
- **ContentView**：iOS平台的登录页面视图，使用SwiftUI实现
- **HomeView**：iOS平台的个人中心视图，显示欢迎信息
- **验证逻辑**：两个平台都实现了输入验证和登录状态管理

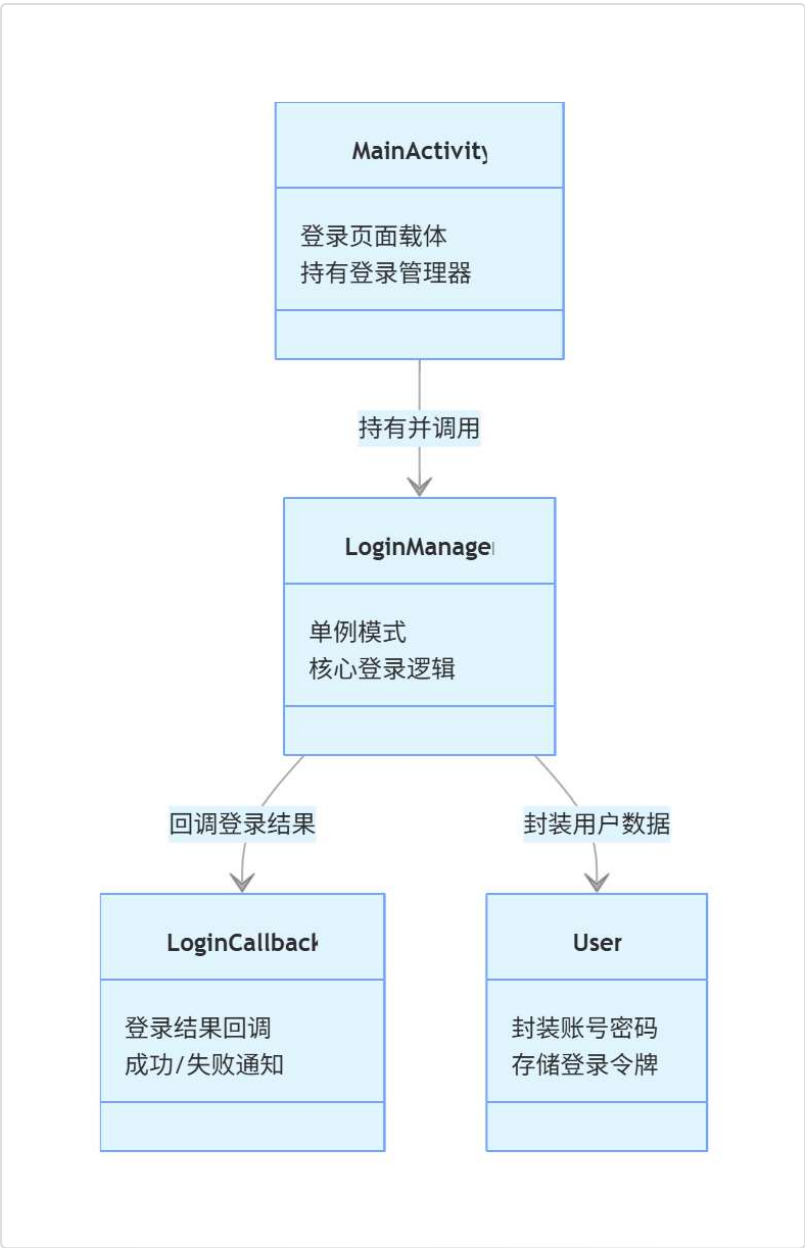


图4. 类图

顺序图

顺序图说明：该顺序图展示了用户登录流程的交互步骤。

- 1. **用户输入：**用户在登录界面输入用户名和密码
- 2. **输入验证：**系统验证输入数据的合法性
- 3. **登录处理：**验证用户名和密码是否正确
- 4. **页面跳转：**登录成功后跳转到个人中心页面
- 5. **数据传递：**将用户名等信息传递到个人中心
- 6. **界面显示：**个人中心显示欢迎信息

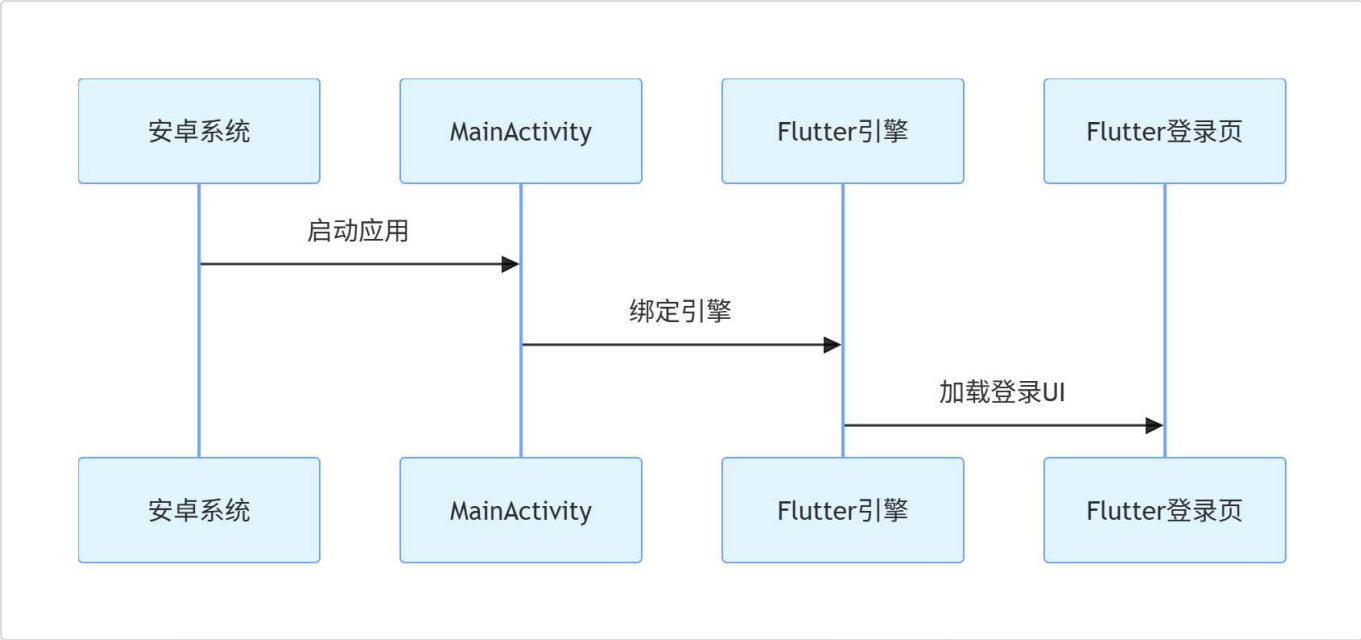


图5. 顺序图

平台差异对比

表4. Android与iOS平台差异对比

对比项	Android	iOS
开发语言	Kotlin	Swift
UI框架	XML/Compose	SwiftUI
页面跳转	Intent	NavigationLink
数据传递	Intent.putExtra	@Binding
布局方式	线性布局/约束布局	Stack/Grid
生命周期	Activity	ViewController

问题与解决

表5. 问题与解决方案

问题描述	解决方案	解决结果
Color.fromRGBO参数缺失	添加第4个参数opacity: 1.0	√ 已解决
Android SDK未配置	运行 flutter config --android-sdk 配置路径	√ 已解决
项目缺少Android配置	运行 flutter create . 重新生成	√ 已解决
手机无法检测	安装Android SDK并接受许可协议	√ 已解决

实验总结

实验收获

- 掌握了Android Kotlin登录页面开发
- 理解了iOS SwiftUI登录实现
- 学会了Activity间数据传递
- 完成了平台差异对比分析

技术要点

- 输入验证系统：使用EditText的验证功能实现了完整的表单验证
- 验证码机制：使用Random类生成4位数字验证码
- 状态管理：通过Activity和Intent管理应用状态
- 页面导航：使用Intent和NavigationLink实现页面跳转
- 数据传递：使用Intent.putExtra和参数传递实现数据共享
- 平台差异：理解Android和iOS平台的开发差异

考核指标

表6. 考核指标评分表

考核项	评分标准	得分
功能完整性	登录功能完整，输入验证正确	25分
代码质量	代码结构清晰，注释完整	20分
页面跳转	Activity/View跳转正常	15分
数据传递	数据传递正确	15分
平台对比	完成平台差异分析	15分
实验报告	报告结构完整，内容详实	10分
总分		100分

教师评价

表7. 教师评价表

项目	评价
实验完成情况	
技术掌握程度	

创新点	
综合评价	
成绩	

教师签名： _____

日期： _____

附录

核心术语

术语	说明	参考链接
Activity	Android应用的基本组件， 代表一个具有用户界面的单一屏幕	Android Activity
Intent	Android中用于组件间通信的消息对象	Android Intent
SwiftUI	Apple开发的声明式UI框架	SwiftUI
ViewController	iOS应用的基本组件， 代表一个具有用户界面的单一屏幕	UIViewController
NavigationLink	SwiftUI中用于页面导航的控件	NavigationLink
Kotlin	JetBrains开发的JVM编程语言	Kotlin
Swift	Apple开发的编程语言	Swift